

ApexPlanet Software Pvt. Ltd.

Cybersecurity Internship — Task 1

Foundation & Environment Setup: Lab Report

Intern:	Tumu Durga Manjusha Krishna Veni
Lab Environment:	Kali Linux (attacker) + Metasploitable2 (target)
Target IP:	192.168.56.101
Attacker IP:	192.168.56.102
Virtualization:	VirtualBox — Host-only network
Tools Used:	VirtualBox, Kali Linux, ip, ping, nmap

This report documents the hands-on execution of Task 1 of the ApexPlanet cybersecurity internship: Foundation & Environment Setup. The objective of this task was to build strong fundamentals in cybersecurity concepts, networking, and cryptography, and to stand up a professional, isolated hacking lab that every later task in this internship builds on top of.

Rather than jumping straight into attack tools, this task deliberately starts at the foundation: understanding core security principles, setting up a safe virtual lab, and getting comfortable with the Linux command line and basic networking — the same groundwork every security professional builds their career on.

1. Cybersecurity Basics

Objective: understand the core principles, threats, and attack vectors that every later technical task in this internship is built to defend against or exploit.

Before touching any tool, a security practitioner needs a mental model of what "security" actually protects and what it's protecting against. This task began conceptually, covering three foundational pillars:

- **The CIA Triad** — Confidentiality (only authorized people can read data), Integrity (data can't be tampered with undetected), and Availability (systems stay up and usable). Nearly every security control that exists maps back to protecting one of these three properties.
- **Threat types** — Phishing (tricking people into giving up credentials), Malware (malicious software), DDoS (overwhelming a system with traffic to take it offline), SQL Injection (manipulating a database through unsanitized input), Brute Force (guessing passwords through repeated attempts), and Ransomware (encrypting a victim's data and demanding payment).
- **Attack vectors** — the actual paths an attacker uses to get in: Social Engineering (manipulating people rather than systems), Wireless Attacks (exploiting Wi-Fi vulnerabilities), and Insider Threats (malicious or careless actions from people who already have legitimate access).

This framework matters because every tool used later in the internship — Nmap, Wireshark, iptables — exists to either exploit one of these threat categories or defend against it. Understanding the "why" before the "how" is what separates someone who runs commands from someone who actually understands security.

2. Lab Environment Setup

Objective: build an isolated, legally safe environment in which scanning and exploitation techniques can be practiced without touching any real-world system.

Every security lab starts with isolation. Running scanning or exploitation tools against a live production network — even a home Wi-Fi router — can trigger false alarms, break services, or cross into illegal territory if it touches a system that isn't owned by the person running the tool. The fix is to build a closed sandbox: virtual machines that only talk to each other on a private network that never reaches the public internet.

VirtualBox was installed as the hypervisor, with two virtual machines configured on a private Host-Only Network adapter:

- **Kali Linux** — a Debian-based distribution that ships pre-loaded with the offensive security toolkit (Nmap, Wireshark, Metasploit, Burp Suite, etc.), configured as the "attacker" machine for the entire internship.
- **Metasploitable2** — an intentionally outdated, vulnerable Ubuntu-based VM published by Rapid7 specifically for training, used here as the "target" machine.



Figure 1: Kali Linux desktop environment, confirming the VM has been successfully installed and is ready for the lab exercises.

With the VM running, basic internet connectivity was verified by opening a browser and loading a search engine — confirming Kali's NAT network adapter was correctly bridging out to the internet for software updates and tool downloads, separate from the private lab network used to talk to Metasploitable2.

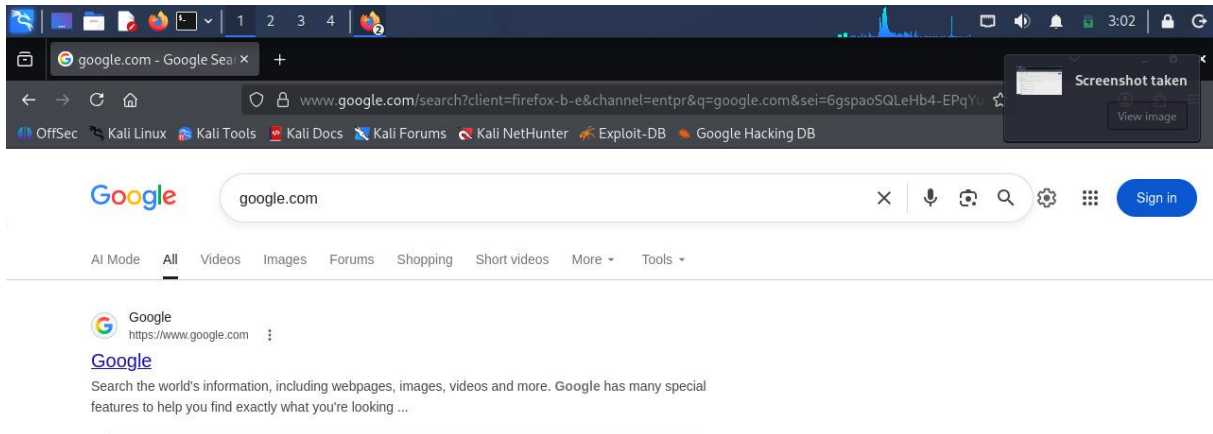


Figure 2: Browser connectivity check from within Kali Linux, confirming internet access is working correctly through the NAT network adapter.

✓ WHAT THIS DEMONSTRATES

Two separate network paths were confirmed working at this stage: a NAT adapter for general internet access, and (set up next) a Host-Only adapter for isolated lab traffic to Metasploitable2 — keeping the dangerous experimentation completely walled off from the real internet.

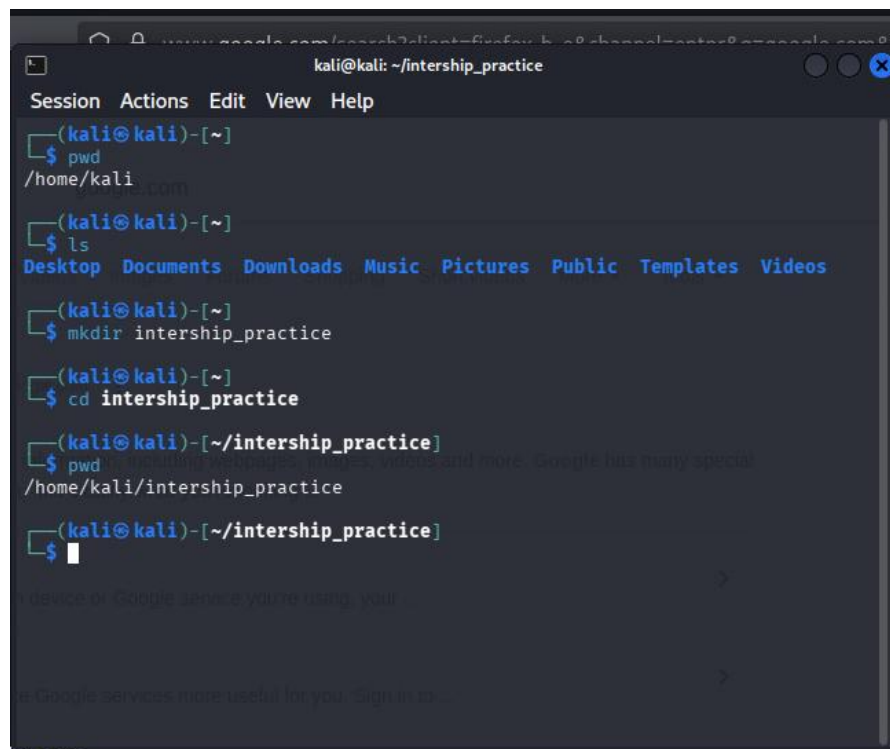
3. Linux Fundamentals

Objective: build comfort with the Linux command line — the primary interface for nearly every security tool used throughout this internship.

Kali Linux, like almost every penetration testing platform, is operated primarily through the terminal rather than a graphical interface. Before any scanning tools were introduced, basic file system navigation was practiced to build that fluency:

- **pwd** ("print working directory") — shows the current location in the file system.
- **ls** — lists the contents of the current directory.
- **mkdir** — creates a new directory.
- **cd** — changes into a directory.

These four commands were chained together to create a dedicated working directory for the internship: starting at the home directory (/home/kali), listing its default folders (Desktop, Documents, Downloads, etc.), creating a new folder named **internship_practice**, and moving into it — confirmed at each step using **pwd**. This directory now serves as the working folder for all internship deliverables and notes.



```
kali@kali: ~/internship_practice
Session Actions Edit View Help
(kali@kali)-[~]
$ pwd
/home/kali
(kali@kali)-[~]
$ ls
Desktop Documents Downloads Music Pictures Public Templates Videos
(kali@kali)-[~]
$ mkdir internship_practice
(kali@kali)-[~]
$ cd internship_practice
(kali@kali)-[~/internship_practice]
$ pwd
/home/kali/internship_practice
(kali@kali)-[~/internship_practice]
$
```

Figure 3: Linux file system navigation: **pwd** confirms the home directory, **ls** lists its contents, **mkdir** creates a new working directory, and **cd** moves into it — verified with a second **pwd**.

Beyond navigation, this task's syllabus also covers file and directory permissions (**chmod**, **chown**), package management (**apt**, **dpkg**), and networking commands — the last of which is demonstrated directly in the next section.

4. Networking Basics

Objective: understand and verify the network configuration of the lab — the IP addressing that makes every later scan and packet capture possible.

With Linux fundamentals in hand, a fresh terminal session was opened to begin the networking exercises — confirming the syllabus topics of IP addressing, subnetting, and basic connectivity testing.

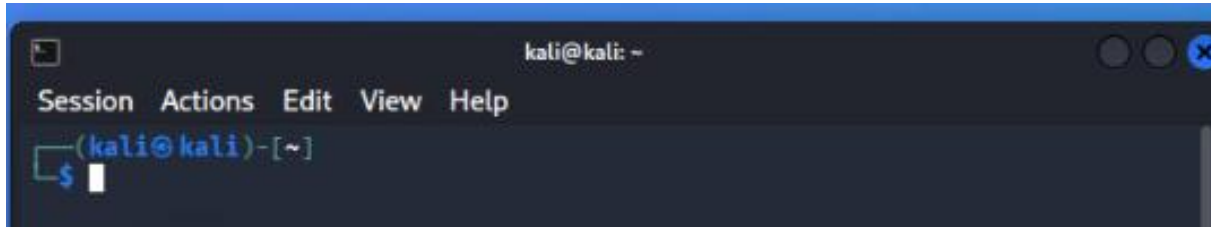


Figure 4: A fresh terminal session opened in Kali, ready for the networking command exercises.

4.1 Inspecting Network Interfaces (ip addr)

The **ip addr** command lists every network interface on the system along with its assigned IP address, confirming exactly how Kali is connected to both the internet and the private lab network. The output showed three interfaces:

- **lo** — the loopback interface (127.0.0.1), used for the machine to talk to itself.
- **eth0** — assigned 10.0.2.15, the NAT adapter providing general internet access.
- **eth1** — assigned 192.168.56.102, the Host-Only adapter placing Kali on the same private subnet as the Metasploitable2 target.

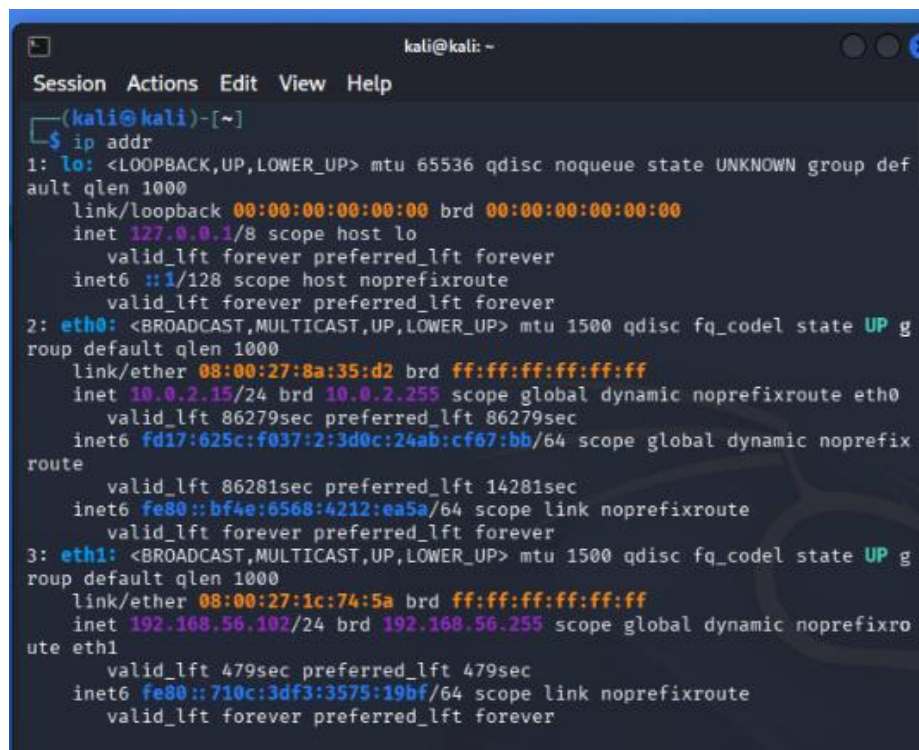
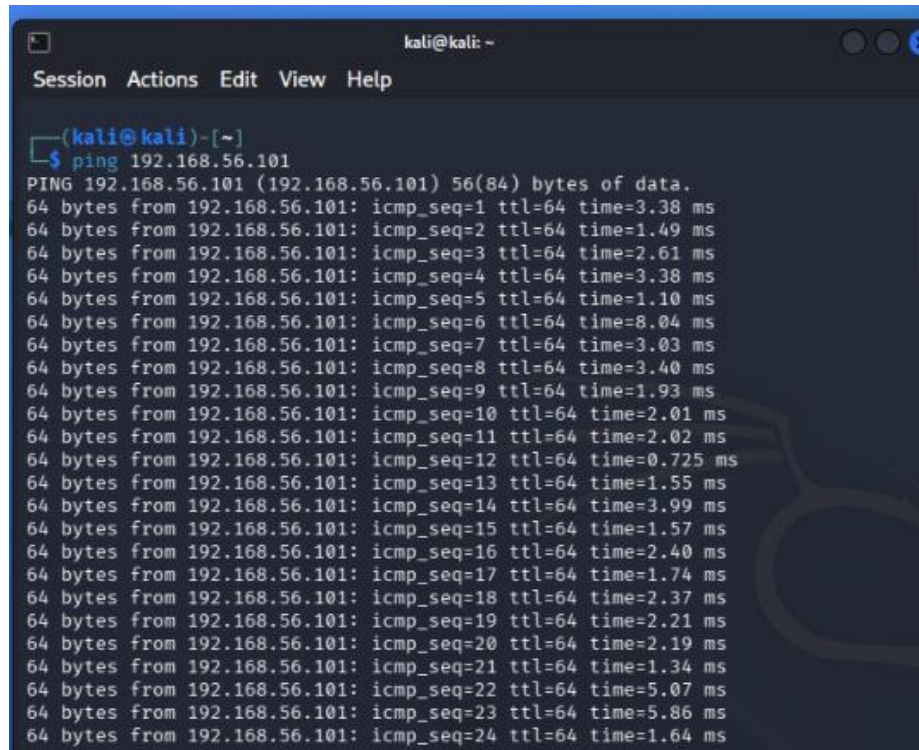


Figure 5: Output of 'ip addr' showing three network interfaces: loopback (lo), the NAT-connected eth0 (10.0.2.15), and the Host-Only eth1 (192.168.56.102) used for lab traffic.

4.2 Testing Connectivity (ping)

With the network interfaces confirmed, **ping 192.168.56.101** was used to test connectivity to the Metasploitable2 target across the private lab subnet. Ping works by sending ICMP Echo Request packets and measuring the Echo Reply that comes back — both confirming the target is reachable and giving a rough sense of network latency (round-trip time, shown in milliseconds for each packet).

The capture below shows 24 consecutive replies, all successful, with round-trip times between roughly 0.7 ms and 8 ms — fast and consistent, exactly what's expected on a local, isolated virtual network with no real-world routing in between.

A screenshot of a Kali Linux terminal window. The window title is 'kali@kali: -'. The menu bar shows 'Session', 'Actions', 'Edit', 'View', and 'Help'. The terminal prompt is '(kali@kali)-[~]'. The user has entered the command '\$ ping 192.168.56.101'. The output shows a series of 24 successful ping replies. Each line indicates '64 bytes from 192.168.56.101: icmp_seq=X ttl=64 time=Y ms', where X ranges from 1 to 24 and Y represents the round-trip time in milliseconds. The times vary, with the highest being 8.04 ms and the lowest being 0.725 ms.

```
kali@kali: -
Session Actions Edit View Help

(kali@kali)-[~]
$ ping 192.168.56.101
PING 192.168.56.101 (192.168.56.101) 56(84) bytes of data.
64 bytes from 192.168.56.101: icmp_seq=1 ttl=64 time=3.38 ms
64 bytes from 192.168.56.101: icmp_seq=2 ttl=64 time=1.49 ms
64 bytes from 192.168.56.101: icmp_seq=3 ttl=64 time=2.61 ms
64 bytes from 192.168.56.101: icmp_seq=4 ttl=64 time=3.38 ms
64 bytes from 192.168.56.101: icmp_seq=5 ttl=64 time=1.10 ms
64 bytes from 192.168.56.101: icmp_seq=6 ttl=64 time=8.04 ms
64 bytes from 192.168.56.101: icmp_seq=7 ttl=64 time=3.03 ms
64 bytes from 192.168.56.101: icmp_seq=8 ttl=64 time=3.40 ms
64 bytes from 192.168.56.101: icmp_seq=9 ttl=64 time=1.93 ms
64 bytes from 192.168.56.101: icmp_seq=10 ttl=64 time=2.01 ms
64 bytes from 192.168.56.101: icmp_seq=11 ttl=64 time=2.02 ms
64 bytes from 192.168.56.101: icmp_seq=12 ttl=64 time=0.725 ms
64 bytes from 192.168.56.101: icmp_seq=13 ttl=64 time=1.55 ms
64 bytes from 192.168.56.101: icmp_seq=14 ttl=64 time=3.99 ms
64 bytes from 192.168.56.101: icmp_seq=15 ttl=64 time=1.57 ms
64 bytes from 192.168.56.101: icmp_seq=16 ttl=64 time=2.40 ms
64 bytes from 192.168.56.101: icmp_seq=17 ttl=64 time=1.74 ms
64 bytes from 192.168.56.101: icmp_seq=18 ttl=64 time=2.37 ms
64 bytes from 192.168.56.101: icmp_seq=19 ttl=64 time=2.21 ms
64 bytes from 192.168.56.101: icmp_seq=20 ttl=64 time=2.19 ms
64 bytes from 192.168.56.101: icmp_seq=21 ttl=64 time=1.34 ms
64 bytes from 192.168.56.101: icmp_seq=22 ttl=64 time=5.07 ms
64 bytes from 192.168.56.101: icmp_seq=23 ttl=64 time=5.86 ms
64 bytes from 192.168.56.101: icmp_seq=24 ttl=64 time=1.64 ms
```

Figure 6: Output of 'ping 192.168.56.101', showing 24 successful Echo replies from the Metasploitable2 target with round-trip times of roughly 0.7-8 ms.

✓ WHAT THIS DEMONSTRATES

Confirming connectivity with ping before running any scanner is standard practice: if the target doesn't reply to a basic ping, there's no point troubleshooting a complex tool like Nmap before first confirming the network path actually works.

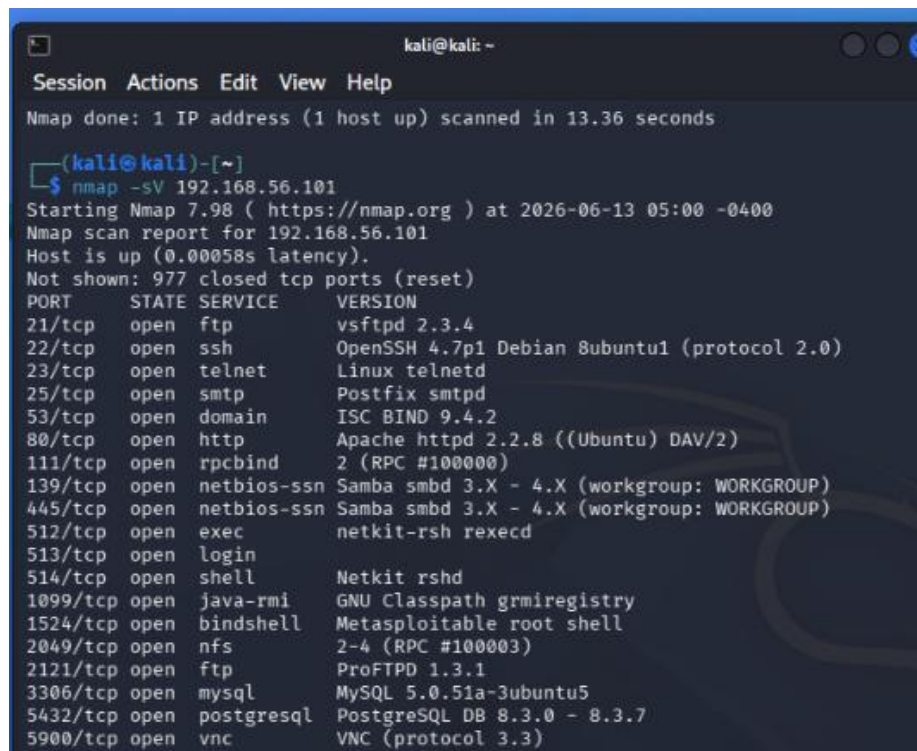
5. Tool Familiarization

Objective: get hands-on with Nmap — the primary scanning tool used throughout this internship — by running an initial scan against the lab target.

With networking basics confirmed, Nmap was introduced for the first time. Two scans were run against the Metasploitable2 target as an initial familiarization exercise, each increasing in depth.

5.1 Service Version Scan (nmap -sV)

nmap -sV 192.168.56.101 identifies not just which ports are open, but the specific software and version listening behind each one. The scan returned a long list of open services — FTP, SSH, Telnet, SMTP, DNS, HTTP, NetBIOS/SMB, MySQL, PostgreSQL, and VNC among them — each tagged with a version number, e.g. vsftpd 2.3.4 on port 21 and OpenSSH 4.7p1 on port 22. This first encounter with Nmap immediately demonstrates why version detection matters: knowing the exact software version is what separates a vague "port is open" observation from genuinely actionable security intelligence.



```
kali@kali: ~  
Session Actions Edit View Help  
Nmap done: 1 IP address (1 host up) scanned in 13.36 seconds  
  
(kali@kali)-[~]  
$ nmap -sV 192.168.56.101  
Starting Nmap 7.98 ( https://nmap.org ) at 2026-06-13 05:00 -0400  
Nmap scan report for 192.168.56.101  
Host is up (0.00058s latency).  
Not shown: 977 closed tcp ports (reset)  
PORT      STATE SERVICE      VERSION  
21/tcp    open  ftp          vsftpd 2.3.4  
22/tcp    open  ssh          OpenSSH 4.7p1 Debian 8ubuntu1 (protocol 2.0)  
23/tcp    open  telnet       Linux telnetd  
25/tcp    open  smtp         Postfix smtpd  
53/tcp    open  domain       ISC BIND 9.4.2  
80/tcp    open  http         Apache httpd 2.2.8 ((Ubuntu) DAV/2)  
111/tcp   open  rpcbind      2 (RPC #100000)  
139/tcp   open  netbios-ssn  Samba smbd 3.X - 4.X (workgroup: WORKGROUP)  
445/tcp   open  netbios-ssn  Samba smbd 3.X - 4.X (workgroup: WORKGROUP)  
512/tcp   open  exec         netkit-rsh rshd  
513/tcp   open  login        Netkit rshd  
514/tcp   open  shell        Netkit rshd  
1099/tcp  open  java-rmi     GNU Classpath grmiregistry  
1524/tcp  open  bindshell    Metasploitable root shell  
2049/tcp  open  nfs          2-4 (RPC #100003)  
2121/tcp  open  ftp          ProFTPD 1.3.1  
3306/tcp  open  mysql        MySQL 5.0.51a-3ubuntu5  
5432/tcp  open  postgresql   PostgreSQL DB 8.3.0 - 8.3.7  
5900/tcp  open  vnc          VNC (protocol 3.3)
```

Figure 7: First Nmap scan of the internship: 'nmap -sV 192.168.56.101', identifying specific service versions on the target, including vsftpd 2.3.4 and OpenSSH 4.7p1.

5.2 Aggressive Scan (nmap -A)

The second scan, **nmap -A 192.168.56.101**, runs Nmap's "aggressive" mode — combining OS detection, service version detection, script scanning (NSE — the Nmap Scripting Engine), and traceroute all in a single command. This produces by far the richest single-scan output covered in this task, and is the closest thing to a complete reconnaissance pass in one command.

The script scanning component in particular surfaced detail far beyond a basic port list. On port 21 (FTP), Nmap's ftp-anon script actually attempted an anonymous login and succeeded — confirming the target allows anyone to log into its FTP server with no credentials at all, and explicitly flagging that the control connection and data transfers both happen in plain text.

```
kali@kali: ~  
Session Actions Edit View Help  
  
(kali@kali)-[~]  
$ nmap -A 192.168.56.101  
Starting Nmap 7.98 ( https://nmap.org ) at 2026-06-13 05:09 -0400  
Nmap scan report for 192.168.56.101  
Host is up (0.0021s latency).  
Not shown: 977 closed tcp ports (reset)  
PORT      STATE SERVICE      VERSION  
21/tcp    open  ftp          vsftpd 2.3.4  
|_ftp-anon: Anonymous FTP login allowed (FTP code 230)  
| ftp-syst:  
|   STAT:  
|   FTP server status:  
|     Connected to 192.168.56.102  
|     Logged in as ftp  
|     TYPE: ASCII  
|     No session bandwidth limit  
|     Session timeout in seconds is 300  
|     Control connection is plain text  
|     Data connections will be plain text  
|     vsFTPd 2.3.4 - secure, fast, stable  
|_End of status  
22/tcp    open  ssh          OpenSSH 4.7p1 Debian 8ubuntu1 (protocol 2.0)  
| ssh-hostkey:  
|   1024 60:0f:cf:e1:c0:5f:6a:74:d6:90:24:fa:c4:d5:6c:cd (DSA)  
|   2048 56:56:24:0f:21:1d:de:a7:2b:ae:61:b1:24:3d:e8:f3 (RSA)  
23/tcp    open  telnet       Linux telnetd  
25/tcp    open  smtp         Postfix smtpd
```

Figure 8: Start of the 'nmap -A' aggressive scan: the ftp-anon script confirms anonymous FTP login is allowed on port 21, with the full FTP server banner and an OpenSSH host key fingerprint for port 22.

■ RISK INSIGHT

Anonymous FTP login being allowed, combined with plain-text control and data connections, means anyone on the network — not just an authenticated user — can browse and potentially exfiltrate files from this server, and any credentials that are used would be visible to anyone capturing traffic with a tool like Wireshark.

Scrolling further through the same scan, Nmap's SMTP script enumerated the legacy SSLv2 cipher suites still supported on port 25, extracted the mail server's SSL certificate details (including an expiry date of April 2010 — over a decade expired), and identified the DNS service on port 53 as ISC BIND 9.4.2 along with the exact Apache HTTP version and page title running on port 80.

```

kali@kali: ~
Session Actions Edit View Help
25/tcp open smtp Postfix smtpd
| sslv2:
|   SSLv2 supported
|   ciphers:
|     SSL2_RC4_128_WITH_MD5
|     SSL2_RC4_128_EXPORT40_WITH_MD5
|     SSL2_DES_64_CBC_WITH_MD5
|     SSL2_RC2_128_CBC_WITH_MD5
|     SSL2_RC2_128_CBC_EXPORT40_WITH_MD5
|     SSL2_DES_192_EDE3_CBC_WITH_MD5
|_
|_smtp-commands: metasploitable.localdomain, PIPELINING, SIZE 10240000, VRFY,
|_ssl-date: 2026-06-13T09:10:02+00:00; -8s from scanner time.
|_ssl-cert: Subject: commonName=ubuntu804-base.localdomain/organizationName=O
|_Not valid before: 2010-03-17T14:07:45
|_Not valid after: 2010-04-16T14:07:45
53/tcp open domain ISC BIND 9.4.2
| dns-nsid:
|_ bind.version: 9.4.2
80/tcp open http Apache httpd 2.2.8 ((Ubuntu) DAV/2)
|_http-title: Metasploitable2 - Linux
|_http-server-header: Apache/2.2.8 (Ubuntu) DAV/2
111/tcp open rpcbind 2 (RPC #100000)
| rpcinfo:
|   program version port/proto service
|   100000 2 111/tcp rpcbind
|   100000 2 111/udp rpcbind

```

Figure 9: Continued 'nmap -A' output: SMTP script results showing outdated SSLv2 cipher suites and an expired SSL certificate (valid until 2010), plus DNS and HTTP service fingerprinting.

```

kali@kali: ~
Session Actions Edit View Help
| 100000 2 111/udp rpcbind
| 100003 2,3,4 2049/tcp nfs
| 100003 2,3,4 2049/udp nfs
| 100005 1,2,3 46809/udp mountd
| 100005 1,2,3 53010/tcp mountd
| 100021 1,3,4 36694/udp nlockmgr
| 100021 1,3,4 59024/tcp nlockmgr
| 100024 1 43219/udp status
|_ 100024 1 50508/tcp status
139/tcp open netbios-ssn Samba smbd 3.X - 4.X (workgroup: WORKGROUP)
445/tcp open netbios-ssn Samba smbd 3.0.20-Debian (workgroup: WORKGROUP)
512/tcp open exec netkit-rsh rshd
513/tcp open login
514/tcp open shell Netkit rshd
1099/tcp open java-rmi GNU Classpath grmiregistry
1524/tcp open bindshell Metasploitable root shell
2049/tcp open nfs 2-4 (RPC #100003)
2121/tcp open ftp ProFTPD 1.3.1
3306/tcp open mysql MySQL 5.0.51a-3ubuntu5
| mysql-info:
|   Protocol: 10
|   Version: 5.0.51a-3ubuntu5
|   Thread ID: 11
|   Capabilities flags: 43564
|   Some Capabilities: SupportsTransactions, Support41Auth, SwitchToSSLAfterH
|   andshake, ConnectWithDatabase, Speaks41ProtocolNew, SupportsCompression, Long
|   ColumnFlag
|   Status: Autocommit

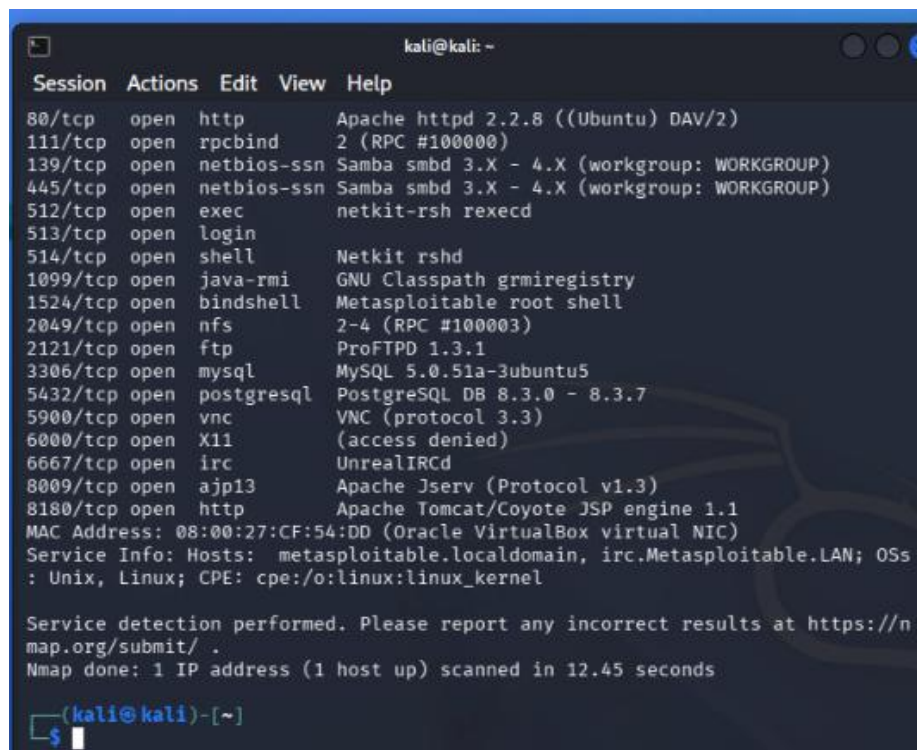
```

Figure 10: Further 'nmap -A' output: full RPC service enumeration (mountd, nlockmgr, status) alongside detailed Samba and MySQL version and protocol information, including supported authentication capabilities.

✓ WHAT THIS DEMONSTRATES

This single nmap -A command produced more actionable detail than several earlier individual scans combined — anonymous FTP access, a decade-expired SSL certificate, and obsolete cipher support were all surfaced automatically by Nmap's scripting engine, illustrating why aggressive mode is often the first scan run in a real assessment, even though it's noisier and slower than a targeted scan.

The scan was run a second time later in the session to verify consistency, completing in 13.22 seconds (compared to 12.45 seconds on the first run) and returning an identical list of 17 open ports and services — confirming the target's configuration, and the scan results themselves, were stable and repeatable.



```
kali@kali: ~
Session Actions Edit View Help
80/tcp open http Apache httpd 2.2.8 ((Ubuntu) DAV/2)
111/tcp open rpcbind 2 (RPC #100000)
139/tcp open netbios-ssn Samba smbd 3.X - 4.X (workgroup: WORKGROUP)
445/tcp open netbios-ssn Samba smbd 3.X - 4.X (workgroup: WORKGROUP)
512/tcp open exec netkit-rsh rshd
513/tcp open login
514/tcp open shell Netkit rshd
1099/tcp open java-rmi GNU Classpath grmiregistry
1524/tcp open bindshell Metasploitable root shell
2049/tcp open nfs 2-4 (RPC #100003)
2121/tcp open ftp ProFTPD 1.3.1
3306/tcp open mysql MySQL 5.0.51a-3ubuntu5
5432/tcp open postgresql PostgreSQL DB 8.3.0 - 8.3.7
5900/tcp open vnc VNC (protocol 3.3)
6000/tcp open X11 (access denied)
6667/tcp open irc UnrealIRCd
8009/tcp open ajp13 Apache Jserv (Protocol v1.3)
8180/tcp open http Apache Tomcat/Coyote JSP engine 1.1
MAC Address: 08:00:27:CF:54:DD (Oracle VirtualBox virtual NIC)
Service Info: Hosts: metasploitable.localdomain, irc.Metasploitable.LAN; OSs
: Unix, Linux; CPE: cpe:/o:linux:linux_kernel

Service detection performed. Please report any incorrect results at https://n
map.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 12.45 seconds

(kali@kali)-[~]
```

Figure 11: Tail end of the nmap -A scan output, listing the complete set of 17 open ports and services identified on the Metasploitable2 target, consistent across repeated scans.

6. Summary of Findings

Pulling every phase together, the table below summarizes what was accomplished and found during this foundational task.

Area	Key Outcome
Conceptual Foundation	Reviewed the CIA Triad, major threat types (phishing, malware, DDoS, SQLi, brute force, ransomware), and key attack vectors (social engineering, wireless, insider threats).
Lab Environment	VirtualBox installed with Kali Linux (attacker) and Metasploitable2 (target) on an isolated Host-Only network, with internet access confirmed via a separate NAT adapter.
Linux Fundamentals	Practiced core navigation commands (pwd, ls, mkdir, cd) and created a dedicated working directory for the internship.
Networking Basics	Verified network interfaces with ip addr (NAT + Host-Only adapters) and confirmed target reachability with ping (24/24 packets successful).
Tool Familiarization	Ran nmap -sV and nmap -A against the target, identifying 17 open services including an anonymous-access FTP server and an SSL certificate expired since 2010.

Closing Note

Task 1 set the groundwork for everything that follows in this internship: a working, isolated lab; comfort with the Linux command line; an understanding of basic networking; and a first hands-on encounter with Nmap that already surfaced real, concrete findings — an open anonymous FTP server and a decade-expired SSL certificate. These same fundamentals — the lab, the terminal fluency, and the scanning tool — are what Task 2 (Network Security & Scanning) builds directly on top of.

All commands and scans in this report were performed exclusively within an isolated VirtualBox lab network against the Metasploitable2 target VM, which is intentionally designed and distributed for this kind of authorized security training.